

Creating a Processor System Lab

Objectives

After completing this lab, you will be able to:

- Understand the steps and directives involved in creating an IP-XACT adapter from a synthesized design in Vivado HLS
- Create a processor system using IP Integrator in Vivado
- Integrate the generated IP-XACT adapter into the created processor system

The design

The design consists of a FIR filter to filter a 4 KHz tone added to CD quality (48 KHz) music. The characteristic of the filter is as follows:

FS=48000 Hz

FPASS1=2000 Hz

FSTOP1=3800 Hz

FSTOP2=4200 Hz

FPASS2=6000 Hz

APASS1=APASS2=1 dB

ASTOP=60 dB

This lab requires you to develop a peripheral core of the designed filter that can be instantiated in a processor system. The processor system will acquire a stereo music stream using an on-board CODEC chip and I2C controller, process it through the designed filter (bandstop filter), and output back to the headphone. **Note: You won't be able to validate this lab on the PYNQ-Z1 board as the board does not have Audio CODEC chip.**

Steps

Create a New Project

Create a new project in Vivado HLS targeting xc7z020clg400-1 device

1. Select **Start > Xilinx Design Tools > Vivado HLS 2018.2 A Getting Started GUI** will appear.
2. In the Getting Started section, click on *Create New Project*. The **New Vivado HLS Project** wizard opens.
3. Click **Browse...** button of the *Location* field, browse to **c:\xup\hls\labs\lab4**, and then click **OK**.
4. For *Project Name*, type **fir.prj** and click **Next**.
5. In the *Add/Remove Files* for the source files, type **fir** as the function name (the provided source file contains the function, to be synthesized, called fir).
6. Click the *Add Files...* button, select **fir.c** and **fir_coef.dat** files from the **c:\xup\hls\labs\lab4** folder, and then click **Open**.
7. Click **Next**.
8. In the *Add/Remove Files* for the testbench, click the *Add Files...* button, select **fir_test.c** file from the **c:\xup\hls\labs\lab4** folder and click **Open**.
9. Click **Next**.

10. In the *Solution Configuration* page, leave *Solution Name* field as *solution1* and make sure the clock period as 10. Leave *Uncertainty* field blank.
11. Click on the Part's Browse button and using the *Parts Specify* option, select **xc7z020clg400-1**.
12. Click **Finish**. You will see the created project in the Explorer view. Expand various sub-folders to see the entries under each sub-folder.
13. Double-click on the *fir.c* under the source folder to open its content in the information pane.

```

1 #include "fir.h"
2
3 void fir (
4     data_t *y,
5     data_t x
6 ) {
7     const coef_t c[N+1]={
8 #include "fir_coef.dat"
9     };
10
11
12     static data_t shift_reg[N];
13     acc_t acc;
14     int i;
15
16     acc=(acc_t)shift_reg[N-1]*(acc_t)c[N];
17     loop: for (i=N-1;i!=0;i--) {
18         acc+=(acc_t)shift_reg[i-1]*(acc_t)c[i];
19         shift_reg[i]=shift_reg[i-1];
20     }
21     acc+=(acc_t)x*(acc_t)c[0];
22     shift_reg[0]=x;
23     *y = acc >> 15;
24 }

```

The design under consideration

The FIR filter expects ***x*** as a sample input and pointer to the computed sample out ***y***. Both of them are defined of data type *data_t*. The coefficients are loaded in array ***c*** of type *coef_t* from the file called **fir_coef.dat** located in the current directory. The sequential algorithm is applied and accumulated value (sample out) is computed in variable *acc* of type *acc_t*.

14. Double-click on the **fir.h** in the outline tab to open its content in the information pane.

```

1 #ifndef _FIR_H_
2 #define _FIR_H_
3 #include "ap_cint.h"
4 #define N 58
5 #define SAMPLES N+10 // just few more samples then number of taps
6 typedef short coef_t;
7 typedef short data_t;
8 typedef int38 acc_t;
9 #endif
10

```

The header file

The header file includes `**ap_cint.h**` so user defined data width (of arbitrary precision) can be used. It also defines number of taps (N), number of samples to be generated (in the testbench), and data types `coef_t`, `data_t`, and `acc_t`. The `coef_t` and `data_t` are short (16 bits). Since the algorithm iterates (multiply and accumulate) over 59 taps, there is a possibility of bit growth of 6 bits and hence `acc_t` is defined as `int38`. Since the `acc_t` is bigger than sample and coefficient width, they have to cast before being used (like in lines 16, 18, and 21 of `fir.c`).

15. Double-click on the **`fir_test.c`** under the testbench folder to open its content in the information pane. Notice that the testbench opens `fir_impulse.dat` in write mode, and sends an impulse (first sample being 0x8000).

Run C Simulation

Run C simulation to observe the expected output.

1. Select **Project > Run C Simulation** or click on the button from the tools bar buttons, and click **OK** in the C Simulation Dialog window. The testbench will be compiled using `apcc` compiler and `csim.exe` file will be generated. The `csim.exe` will then be executed and the output will be displayed in the console view.

```
INFO: [SIM 2] ***** CSIM start *****
INFO: [SIM 4] CSIM will launch GCC as the compiler.
  Compiling(apcc) D:/HLS_2018.2update/HLS/labsource/labs/lab4/fir_test.c in debug mode
INFO: [HLS 200-10] Running 'C:/Xilinx/Vivado/2018.2/bin/unwrapped/win64.o/apcc.exe'
INFO: [HLS 200-10] For user 'RIHL' on host 'rihl' (Windows NT amd64 version 6.2) on Wed Aug 01 14:45:10 +0800 2018
INFO: [HLS 200-10] In directory 'C:/xup/hls/labs/lab4/fir.prj/solution1/csim/build'
INFO: [APCC 202-3] Tmp directory is apcc_db
INFO: [APCC 202-1] APCC is done.
  Compiling(apcc) D:/HLS_2018.2update/HLS/labsource/labs/lab4/fir.c in debug mode
INFO: [HLS 200-10] Running 'C:/Xilinx/Vivado/2018.2/bin/unwrapped/win64.o/apcc.exe'
INFO: [HLS 200-10] For user 'RIHL' on host 'rihl' (Windows NT amd64 version 6.2) on Wed Aug 01 14:45:15 +0800 2018
INFO: [HLS 200-10] In directory 'C:/xup/hls/labs/lab4/fir.prj/solution1/csim/build'
INFO: [APCC 202-3] Tmp directory is apcc_db
INFO: [APCC 202-1] APCC is done.
  Generating csim.exe
0 -32768 378
1 0 73
2 0 -27
3 0 -170
4 0 -298
5 0 -352
6 0 -302
7 0 -168
8 0 -14
9 0 80
10 0 64
11 0 -53
12 0 -186
13 0 -216
```

Initial part of the generated output in the Console view

You should see the filter coefficients being computed.

Synthesize the Design

Synthesize the design with the defaults. View the synthesis results and answer the question listed in the detailed section of this step.

1. Select **Solution > Run C Synthesis > Active Solution** to start the synthesis process.

- When synthesis is completed, several report files will become accessible and the Synthesis. Results will be displayed in the information pane.
- The *Synthesis Report* shows the performance and resource estimates as well as estimated latency in the design.
- Using scroll bar on the right, scroll down into the report and answer the following question.

Question 1

Estimated clock period:

Worst case latency:

Number of DSP48E used:

Number of BRAMs used:

Number of FFs used:

Number of LUTs used:

- The report also shows the top-level interface signals generated by the tools.

Interface

Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	fir	return value
ap_rst	in	1	ap_ctrl_hs	fir	return value
ap_start	in	1	ap_ctrl_hs	fir	return value
ap_done	out	1	ap_ctrl_hs	fir	return value
ap_idle	out	1	ap_ctrl_hs	fir	return value
ap_ready	out	1	ap_ctrl_hs	fir	return value
y	out	16	ap_vld	y	pointer
y_ap_vld	out	1	ap_vld	y	pointer
x	in	16	ap_none	x	scalar

Generated interface signals

You can see the design expects x input as 16-bit scalar and outputs y via pointer of the 16-bit data. It also has ap_vld signal to indicate when the result is valid.

Add PIPELINE directive to the loop and re-synthesize the design. View the synthesis results.

- Make sure that the *fir.c* is open in the information view.
- Select the **Directive** tab, and apply the **PIPELINE** directive to the loop.
- Select **Solution > Run C Synthesis > Active Solution** to start the synthesis process.
- When synthesis is completed, the *Synthesis Results* will be displayed in the information pane.
- Note that the latency has reduced to 62 clock cycles. The DSP48 and BRAM consumption remains same; however, LUT and FF consumptions have slightly increased.

Run RTL/C CoSimulation

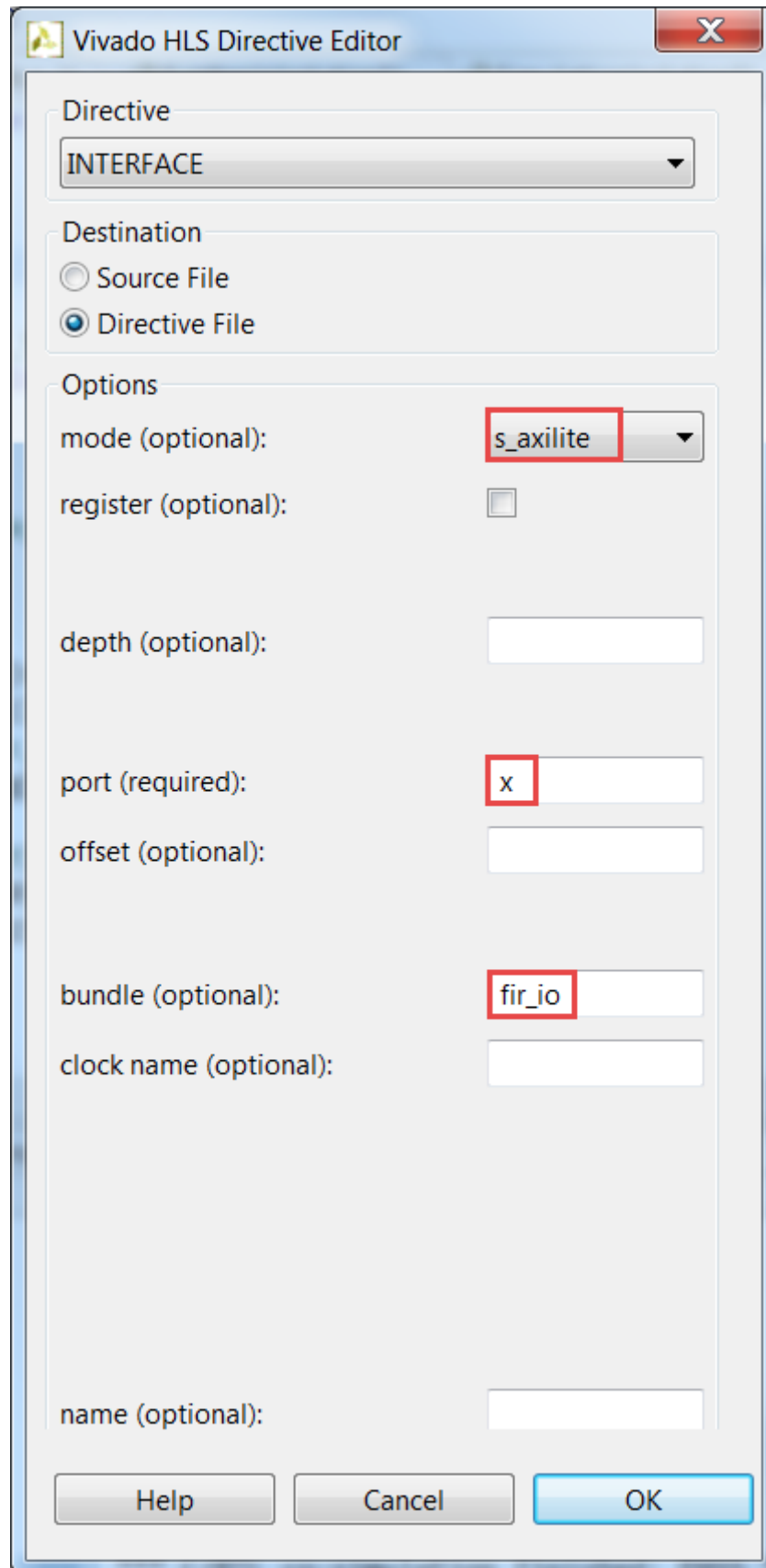
Run the RTL/C Co-simulation, selecting Verilog. Verify that the simulation passes.

1. Select **Solution > Run C/RTL Co-simulation** or click on the button to open the dialog box so the desired simulations can be run.
2. Select the *Verilog* option and click **OK**. The Co-simulation will run, generating and compiling several files, and then simulating the design. In the console window you can see the progress. When done the RTL Simulation Report shows that it was successful and the latency reported was 62.

Setup IP-XACT Adapter

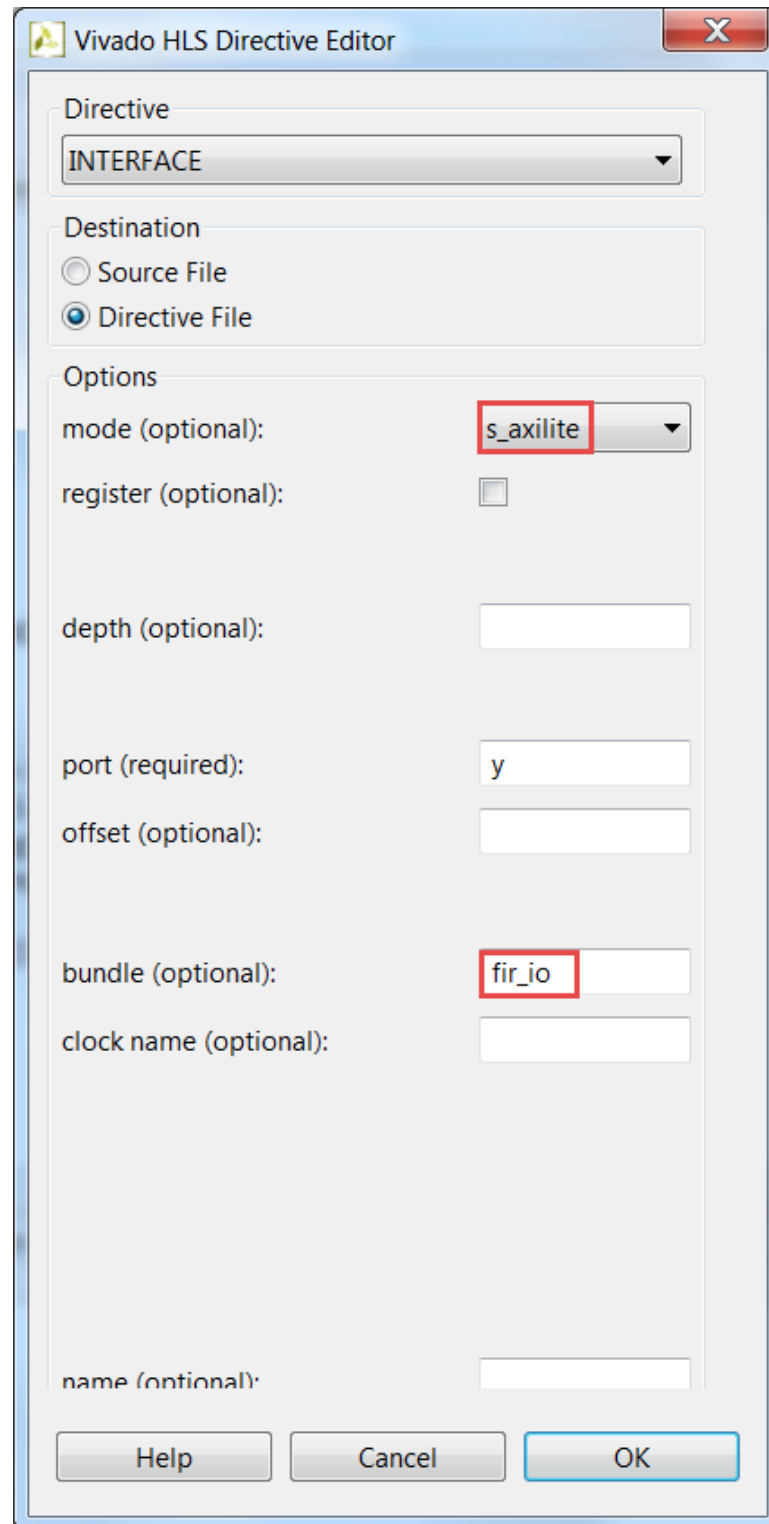
Add INTERFACE directive to create AXI4LiteS adapters so IP-XACT adapter can be generated during the RTL Export step.

1. Make sure that **fir.c** file is open and in focus in the information view.
2. Select the **Directive** tab.
3. Right-click *x*, and click on **Insert Directive...**
4. In the Vivado HLS Directive Editor dialog box, select **INTERFACE** using the drop-down button.
5. Click on the button beside *mode (optional)*. Select **s_axilite**.
6. In the *bundle (optional)* field, enter **fir_io** and click **OK**.



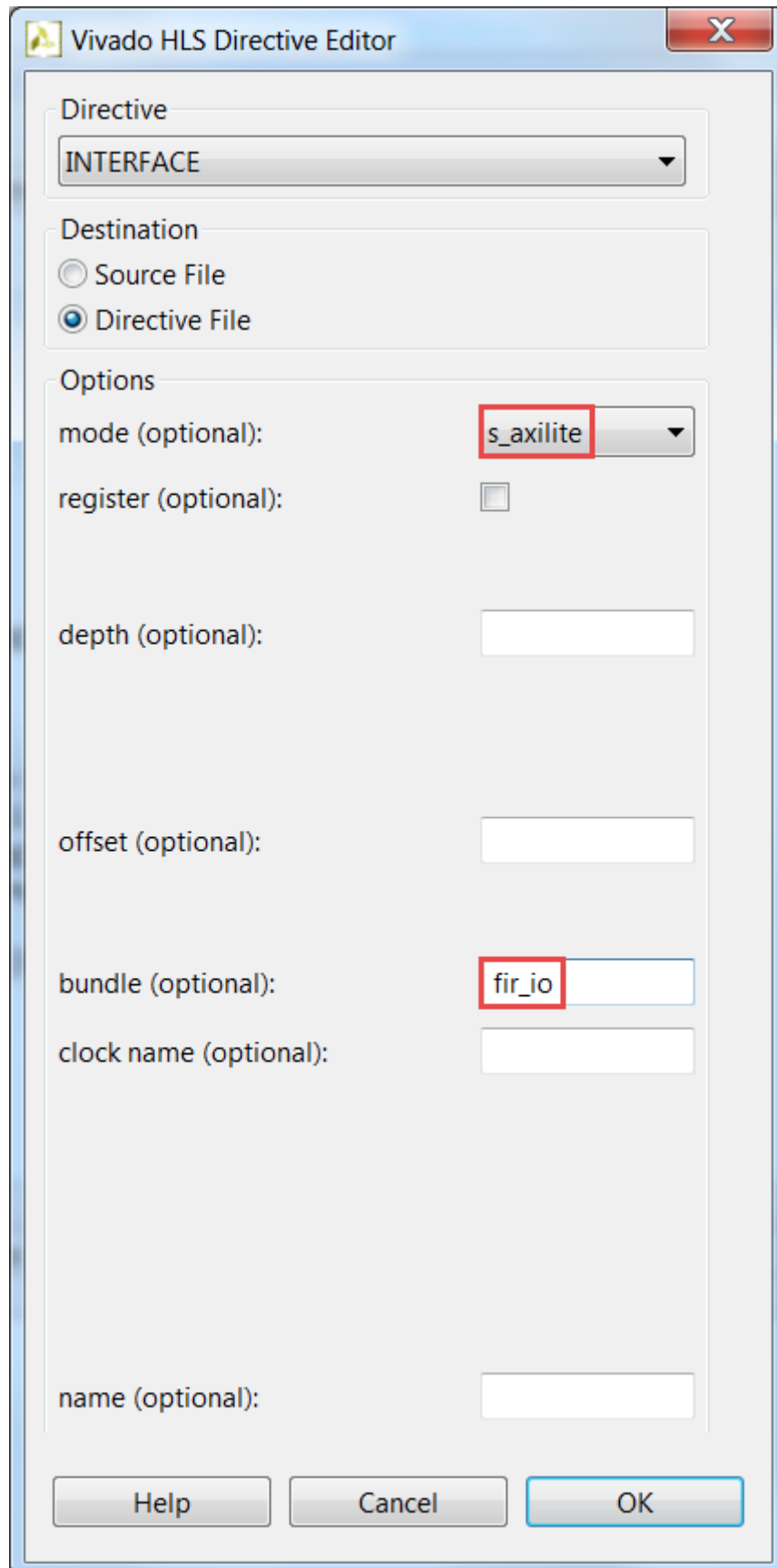
Selecting the AXI4LiteS adapter and naming bundle

7. Similarly, apply the **INTERFACE** directive (including bundle) to the y output.



Applying bundle to assign y output to AXI4Lite adapter

8. Apply the **INTERFACE** directive to the *top-level module fir* to include `ap_start`, `ap_done`, and `ap_idle` signals as part of bus adapter (the variable name shown will be `return`). Include the bundle information too.



The image shows the 'Vivado HLS Directive Editor' dialog box. It has a title bar with a Vivado icon, the text 'Vivado HLS Directive Editor', and a close button (X). The dialog is divided into several sections: 'Directive' with a dropdown menu set to 'INTERFACE'; 'Destination' with two radio buttons, 'Source File' (unselected) and 'Directive File' (selected); 'Options' with several fields: 'mode (optional):' is a dropdown menu with 's_axilite' selected and highlighted by a red box; 'register (optional):' is an unchecked checkbox; 'depth (optional):' is an empty text field; 'offset (optional):' is an empty text field; 'bundle (optional):' is a text field with 'fir_io' entered and highlighted by a red box; 'clock name (optional):' is an empty text field; and 'name (optional):' is an empty text field. At the bottom are three buttons: 'Help', 'Cancel', and 'OK'.

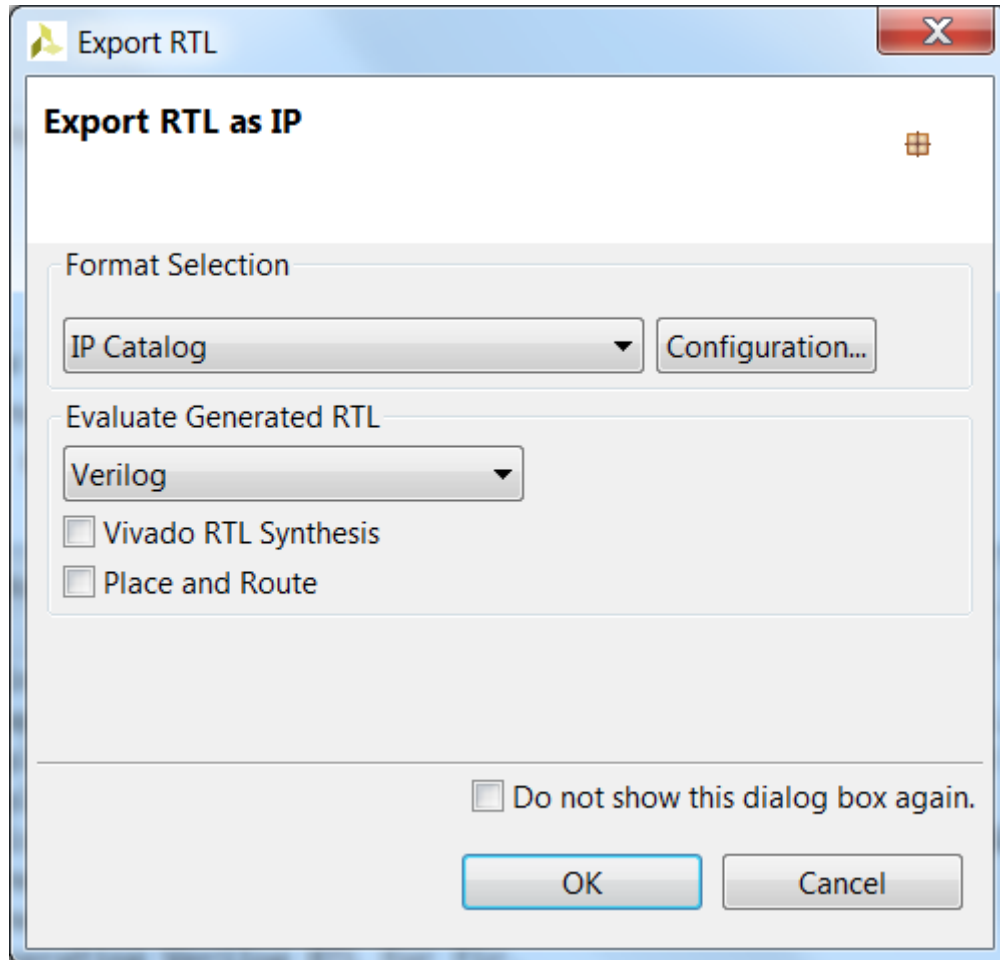
Applying bundle to assign function control signals to AXI4Lite adapter

Note that the above steps will create address maps for `x`, `y`, `ap_start`, `ap_valid`, `ap_done`, and `ap_idle`, which can be accessed via software. Alternately, `ap_start`, `ap_valid`, `ap_done`, `ap_idle` signals can be generated as separate ports on the core by not applying `*RESOURCE*` directive to the top-level module `fir`. These ports will then have to be connected in a processor system using available GPIO IP.

Generate IP-XACT Adapter

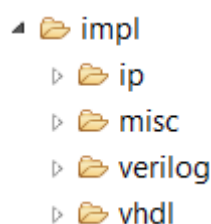
Re-synthesize the design as directives have been added. Run the RTL Export to generate the IP-XACT adapter.

1. Since the directives have been added, it is safe to re-synthesize the design. Select **Solution > Run C Synthesis > Active Solution** Check the Interface summary at the bottom of the Synthesis report to see the interface that has been created.
2. Once the design is synthesized, select **Solution > Export RTL** to open the dialog box so the desired IP can be generated. An **Export RTL Dialog** box will open.



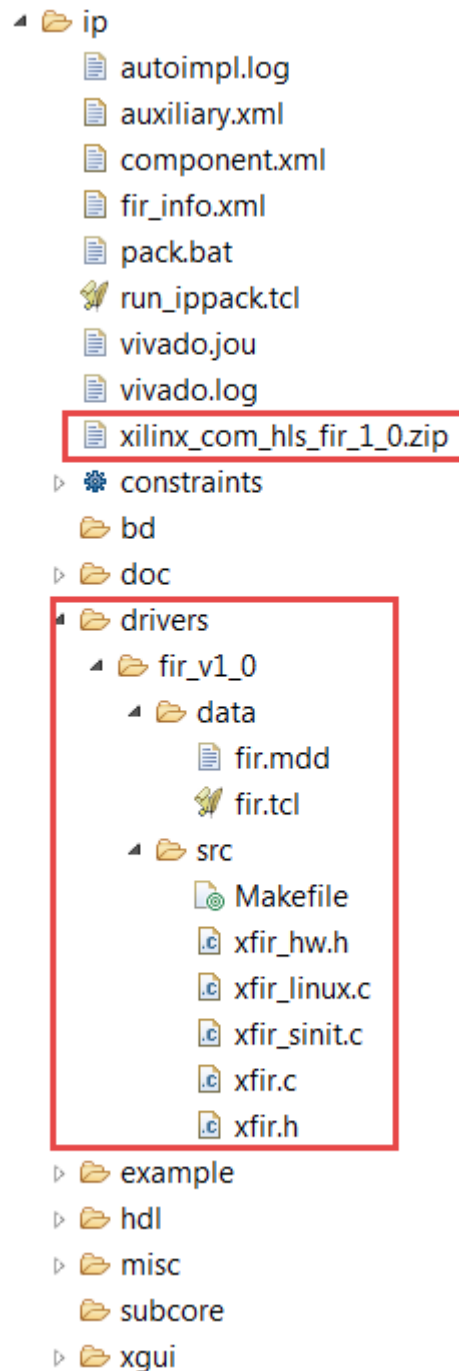
Export RTL Dialog

3. Click **OK** to generate the IP-XACT adapter.
4. When the run is completed, expand the **impl** folder in the Explorer view and observe various generated directories, such as ip, misc, verilog and vhdl.



IP-XACT adapter generated

Expand the ip directory and observe several files and sub-directories. One of the sub-directory of interest is the drivers directory which consists of header, c, tcl, mdd, and makefile files. Another file of interest is the zip file, which is the ip repository file that can be imported in an IP Integrator design



Adapter's drivers directory

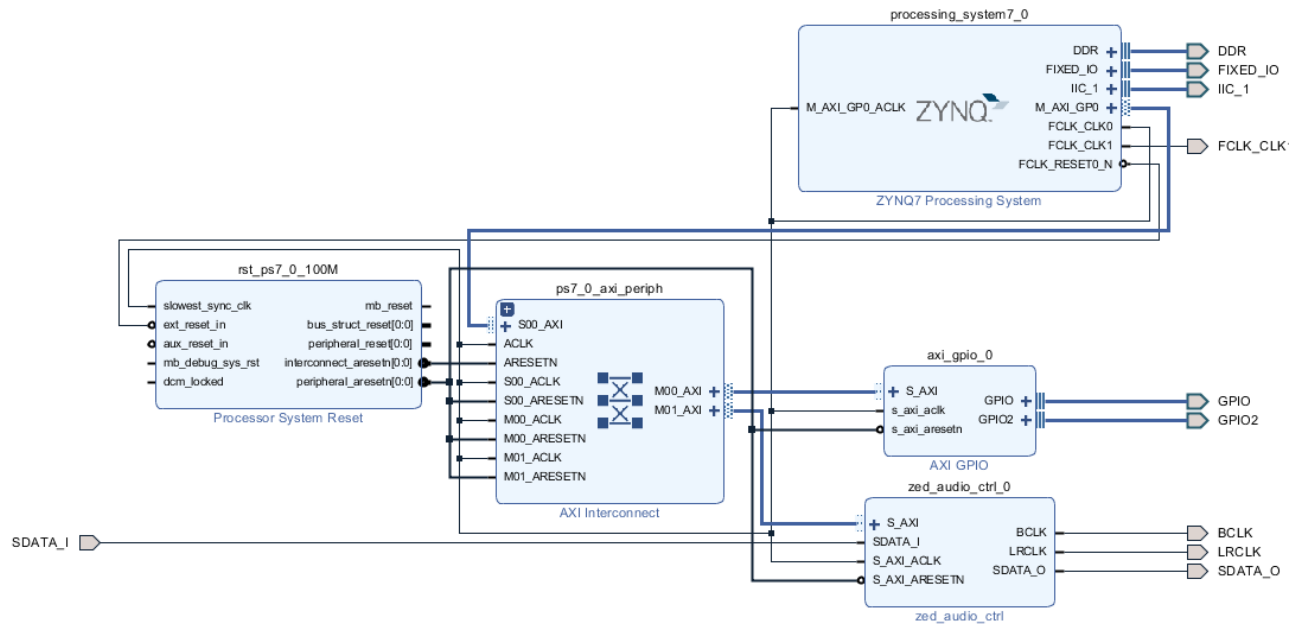
5. Close Vivado HLS by selecting **File > Exit**.

Create a Vivado Project

Launch Vivado Tcl Shell and run the provided tcl script to create an initial system targeting the PYNQ-Z2 board.

1. Open *Vivado Tcl Shell* by selecting **Start > Xilinx Design Tools > Vivado 2018.2 Tcl Shell**
2. In the shell window, change the directory to **c:/xup/hls/labs/lab4** using the `cd` command.

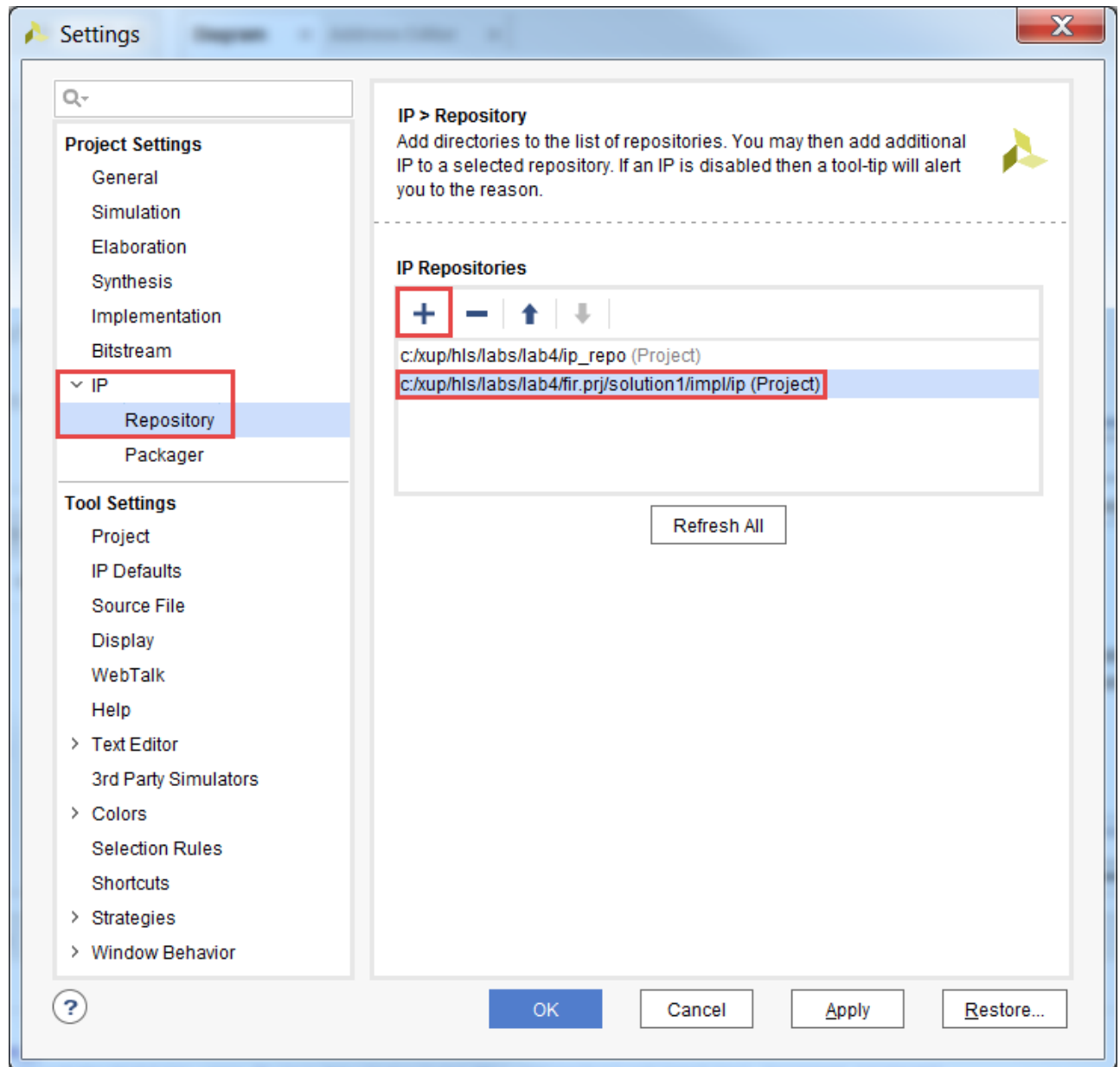
- Run the provided script file to create an initial system having zed_audio_ctrl and GPIO peripherals by typing the following command: `source pynq_z2_audio_project_create.tcl` The script will be run and the initial system, shown below, will be created.



Block design made for Pynq

Add the HLS IP to the IP Catalog

- Select **Flow Navigator > Project Manager > Settings**
- Expand **IP > Repository** in the left pane.
- Click the **+** button (The lab4/ip_repo directory has already been added). Browse to **c:\xup\hls\labs\lab4\fir.prj\solution1\impl\ip** and click **Select**. The directory will be scanned and added in the IP Repositories window, and one IP entry will be detected.
- Click **OK**.



Setting path to IP Repositories

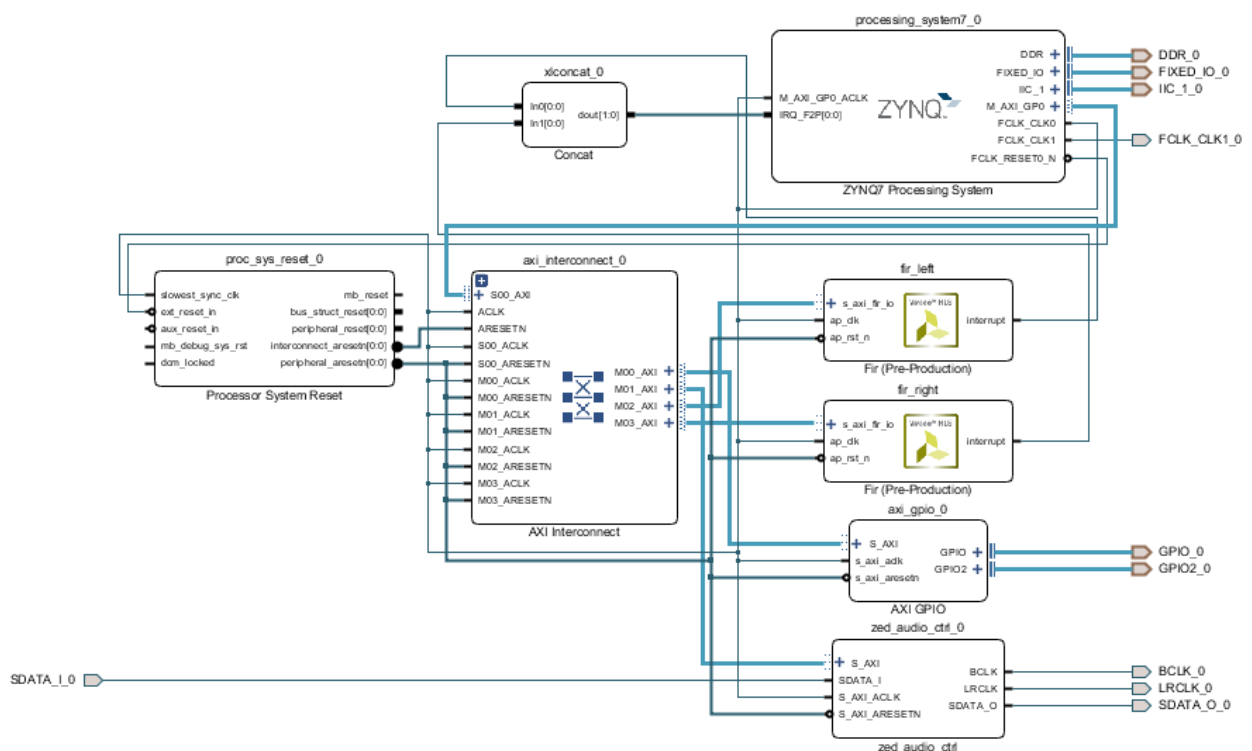
5. Click **OK** to accept the settings

Instantiate `fir_top` core twice, one for each side channel, into the processing system naming the instances as `fir_left` and `fir_right`.

1. Click the *Add IP* icon (plus sign icon) and search for **Fir** in the catalog by typing **Fir** and double-click on the **Fir** entry to add an instance. Notice that the added IP has HLS logo in it indicating that this was created by Vivado HLS.
2. Select the added instance in the diagram, and change its instance name to **fir_left** by typing it in the *Name* field of the *Block Properties* form in the left.
3. Similarly, add another instance of the HLS IP, and naming it **fir_right**.
4. Click on **Run Connection Automation**, and select **All Automation**.
5. Click on **/fir_left/s_axi_fir_io** and **/fir_right/ s_axi_fir_io**. and verify that they will both be connected to the M_AXI_GP0, and click **OK**.

Enable the PS-PL Interrupt ports > IRQ_F2P ports. Add an instance of concat IP with two single-bit input ports. Connect input ports to the interrupt ports of the two FIR instances and the output port to the IRQ_F2P port of the processing_system7_0 instance.

1. Double-click on the *processing_system7_0* instance to open the re-customization form.
2. Select the *Interrupt* entry in the left pane, click on the *Fabric Interrupts* check box in the right.
3. Expand the **Fabric Interrupts > PL-PS Interrupt Ports > IRQ_F2P** entry in the right, and click the check-box of *IRQ_F2P[15:0]*.
4. Click **OK**.
5. Add an instance of the *concat* IP.
6. Connect the interrupt port of each of the FIR instances to the two input ports of the *xlconcat_0* instance.
7. Connect the output port of the *xlconcat_0* instance to the **IRQ_F2P** port of the *processing_system7_0* instance. At this stage the design should look like shown below (you may have to click the regenerate button).



Verify addresses and validate the design. Generate the system_wrapper file, and add the provided Xilinx Design Constraints (XDC).

1. Click on the **Address Editor**, and expand the **processing_system7_0 > Data** if necessary. The generated address map should look like as shown below.

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
axi_gpio_0	S_AXI	Reg	0x4120_0000	64K	0x4120_FFFF
zed_audio_ctrl_0	S_AXI	reg0	0x43C0_0000	64K	0x43C0_FFFF
fir_left	s_axi_fir_io	Reg	0x43C1_0000	64K	0x43C1_FFFF
fir_right	s_axi_fir_io	Reg	0x43C2_0000	64K	0x43C2_FFFF

Generated address map

2. Run *Design Validation* (**Tools > Validate Design**) and verify there are no errors
3. In the sources view, right-click on the block diagram file, **system.bd**, and select **Create HDL Wrapper** to update the HDL wrapper file. When prompted, click OK with the Let Vivado manage wrapper and auto-update option.
4. Click **Add Sources** in the Flow Navigator pane, select **Add or Create Constraints**, and click **Next**.
5. Click the *Add Files* button, browse to the **c:\xup\hls\labs\lab4** folder, select **pynq_z2_audio_constraints.xdc**.
6. Click *Copy constraints files into project* and then click **Finish** to add the file.
7. Click on the **Generate Bitstream** in the Flow Navigator to run the synthesis, implementation, and bitstream generation processes.
8. Click **Save, Yes**, and **OK** if prompted to start the process.
9. When the bit generation is completed, a selection box will be displayed with *Open Implemented Design* option selected. Click **Cancel**.

Export to SDK and create Application Project

Export the hardware along with the generated bitstream to SDK.

1. Select **File > Export > Export Hardware...**
2. Make sure that *Include Bitstream* option is selected and click **OK**, leaving the target directory set to local project directory.
3. Select **File > Launch SDK**
4. Click **OK**.
5. In SDK, select **File > New > Board Support Package**.
6. Click **Finish** with the default settings (with **standalone operating system**). This will open the *Software Platform Settings* form showing the OS and libraries selections.
7. Click **OK** to accept the default settings, as we want to create a *standalone_bsp_0* software platform project without requiring any additional libraries support. The library generator will run in the

background and will create **xparameters.h** file in the

C:\xup\hls\labs\lab4\audio\audio.sdk\standalone_bsp_0\ps7_cortexa9_0\include directory.

8. Select **File > New > Application Project**.
9. Enter **TestApp** as the *Project Name*, and for Board Support Package, choose **Use existing** (standalone_bsp should be the only option)
10. Click **Next**, select **Empty Application** and click **Finish**
11. Select *TestApp* in the project view, right-click the *src* folder, and select **Import**.
12. Expand **General** category and double-click on **File System**.
13. Browse to **c:\xup\hls\labs\lab4** folder and click **OK**.
14. Select both **pynq_z2_testapp.c** and **pynq_z2_audio.h** and click **Finish** to add the file to the project.
The program should compile successfully.

Verify the Design in Hardware

Connect a micro-usb cable between a PC and the JTAG port of the board. Connect an audio patch cable between the Line In jack and the speaker (headphone) out jack of a PC. Connect a headphone to the Line Out jack on the board. Power On the board.

1. Connect a micro-usb cable between a PC and the JTAG port of the board.
2. Connect an audio patch cable between the **Line In** jack and the **speaker out** (headphone) jack of a PC.
3. Connect a headphone to the *HP+MIC* jack on board. Power **ON** the board.
4. Select **Xilinx > Program FPGA**.
5. Make sure that the **system_wrapper.bit** bitstream is selected and the **BMM file** field is blank.
6. Click **Program**. This will configure the FPGA.
7. Double-click **corrupted_music_4KHz.wav** or some other wave file of interest to play it using the installed media player. Place it in the continuous play mode.
8. Right-click on the *TestApp* in the Project Explorer pane and select **Run As > Launch On Hardware (System Debugger)**. The program will be downloaded and run. If you want to listen to corrupted signal then set the **SW0 OFF**. To listened the filtered signal set the **SW0 ON**.
9. When done, power OFF the board.
10. Exit SDK and Vivado using **File > Exit**.

Conclusion

In this lab, you added INTERFACE directive to create an IP-XACT adapter. You generated the IP-XACT adapter during the implementation phase. You then created a processor system using IP Integrator, integrated the generated IP-XACT adapter, and tested the system with the provided application.

Answer

Answers for question 1:

Estimated clock period: **8.70 ns**

Worst case latency: **174**

Number of DSP48E used: **3**

Number of BRAMs used: **0**

Number of FFs used: **167**

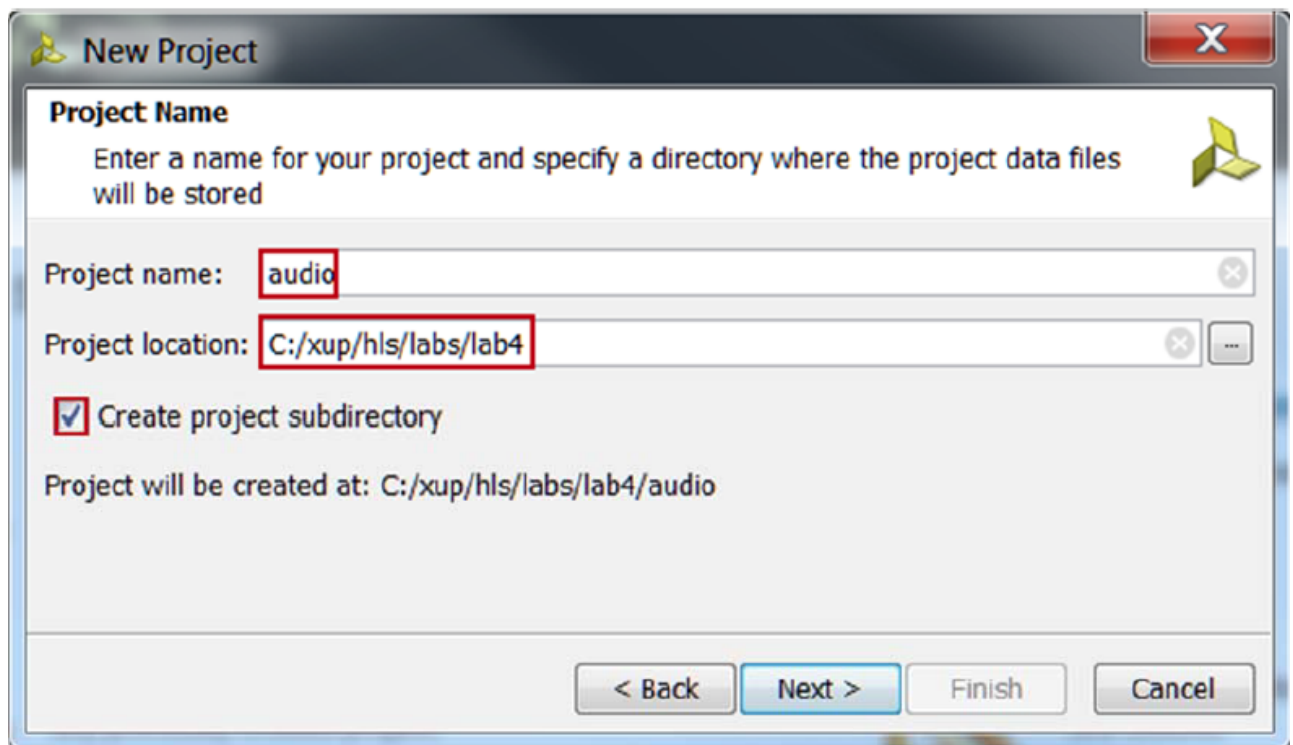
Number of LUTs used: **154**

Appendix

Create a Project using Vivado GUI

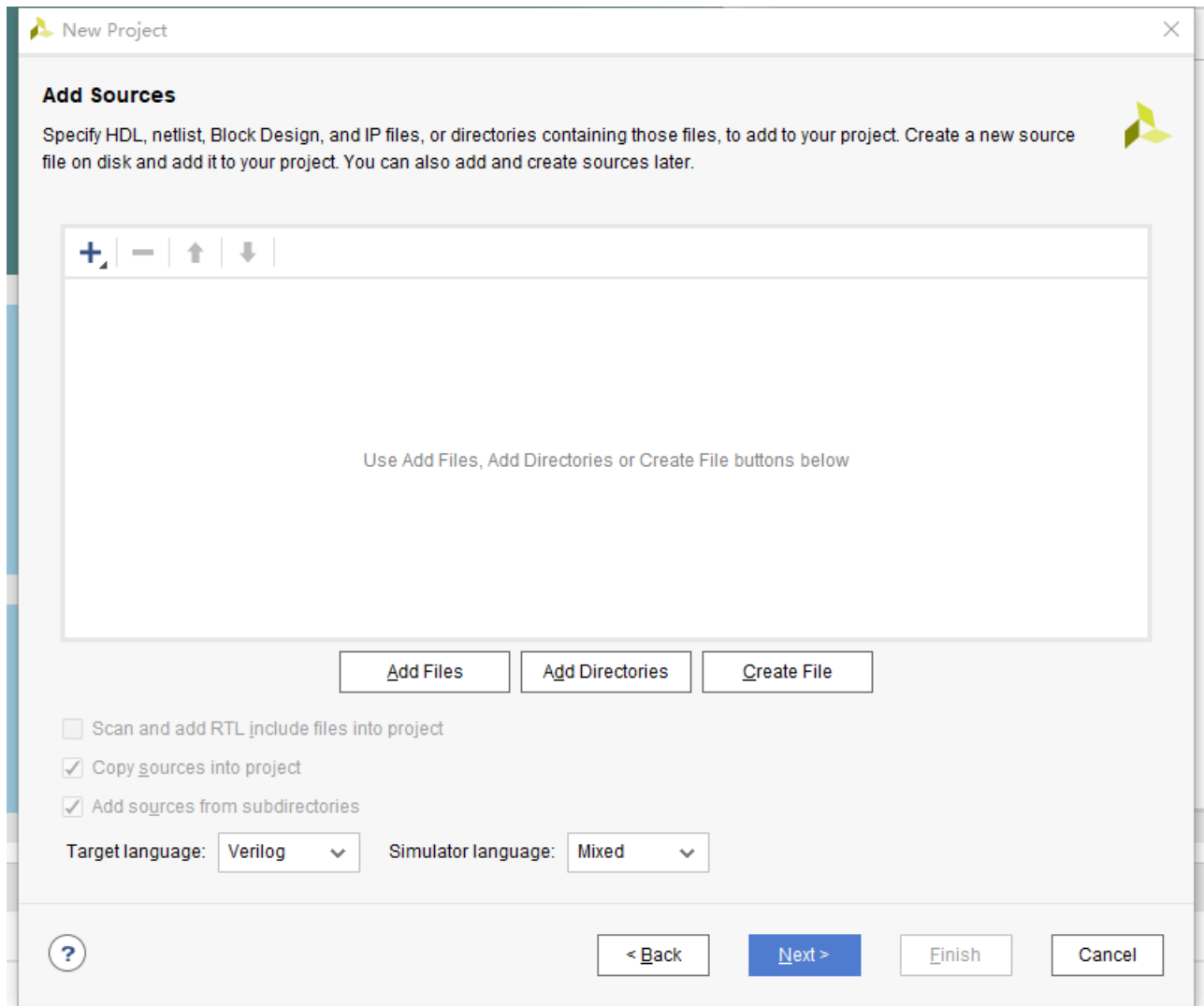
Launch Vivado and create an empty project targeting the Pynq (xc7z020clg400-1)

1. Open Vivado by selecting **Start > Xilinx Design Tools > Vivado 2018.2**
2. Click **Create New Project** to start the wizard. You will see the Create a New Vivado Project dialog box. Click **Next**.
3. Click the Browse button of the Project Location field of the New Project form, browse to **c:\xup\hls\labs\lab4**, and click **Select**.
4. Enter **audio** in the Project Name field. Make sure that the *Create Project Subdirectory* box is checked. Click Next.



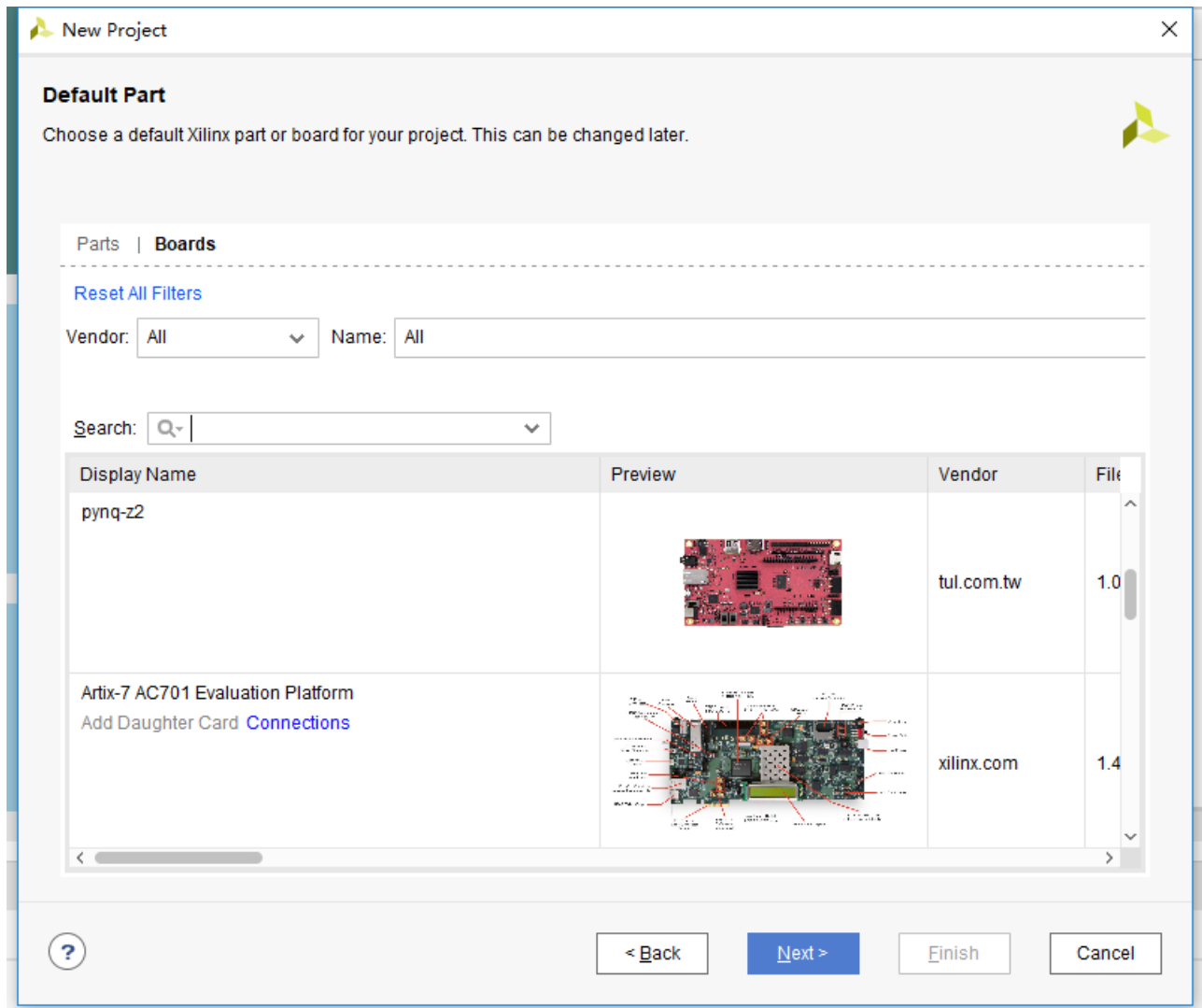
Project Name entry

5. Select *RTL Project* in the Project Type form, and click **Next**.
6. Select *Verilog* as the Target language and Simulator Language in the Add Sources form, and click **Next**.



Add sources to new project

7. Click **Next** two times to skip *Adding Existing IP* and *Add Constraints* dialog boxes
8. In the *Default Part* form, select Boards, and select pynq-z2. Click **Next**.



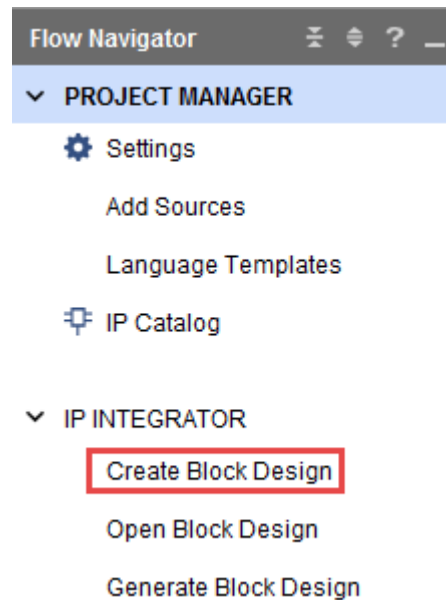
Boards and Parts selection

9. Check the Project Summary and click **Finish** to create an empty Vivado project.

Creating the System Using the IP Integrator

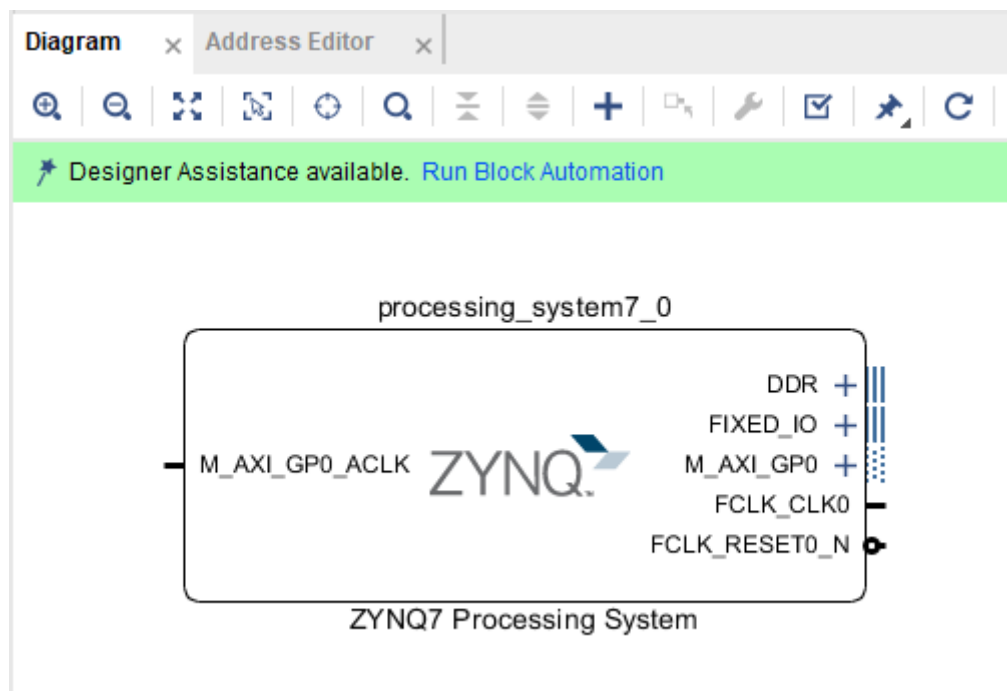
Use the IP Integrator to create a new Block Design, and generate the ARM Cortex-A9 processor based hardware system.

1. In the *Flow Navigator*, click **Create Block Design** under IP Integrator



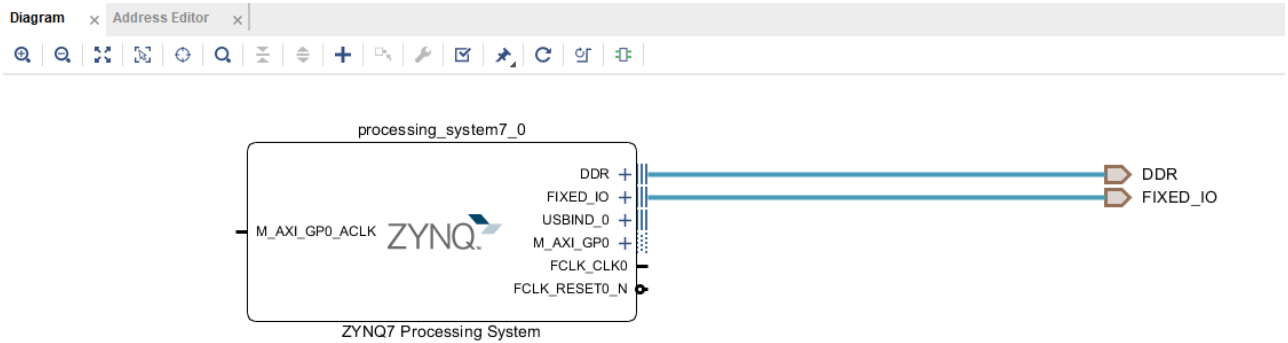
Create IP Integrator Block Diagram

2. Enter **system** for the design name and click **OK**.
3. IP from the catalog can be added in different ways. Click on *Add IP* in the message at the top of the Diagram panel, or click the Add IP icon in the block diagram side bar, press Ctrl + I, or right-click anywhere in the Diagram workspace and select Add IP.
4. Once the IP Catalog is open, type "zy" into the Search bar, find and double click on **ZYNQ7 Processing System** entry, or click on the entry and hit the Enter key to add it to the design.



The Zynq IP Block

5. Notice the message at the top of the Diagram window that Designer Assistance available. Click on **Run Block Automation** and select `/processing_system7_0`
6. Click **OK** when prompted to run automation.

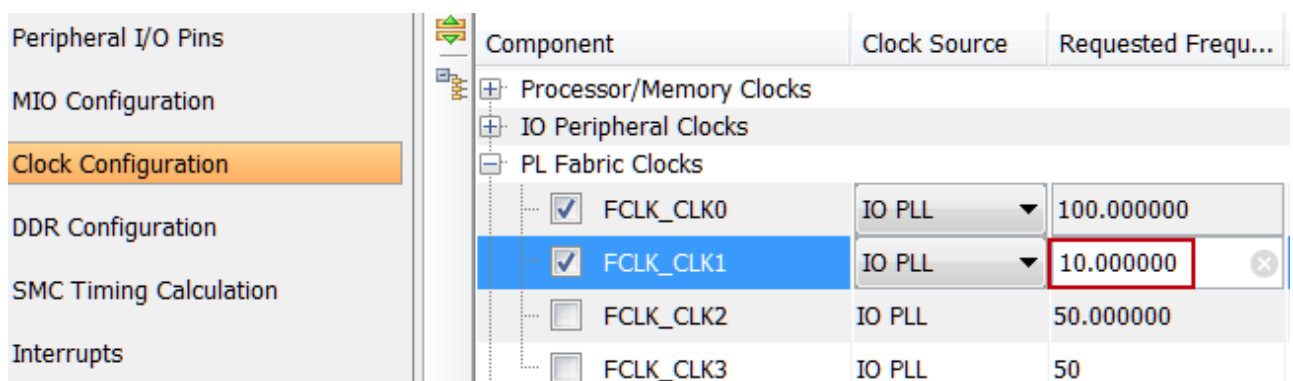


Zynq Block with DDR and Fixed IO ports

7. In the block diagram, double click on the Zynq block to open the Customization window for the Zynq processing system. A block diagram of the Zynq should now be open, showing various configurable blocks of the Processing System. At this stage, the designer can click on various configurable blocks (highlighted in green) and change the system configuration.

Configure I/O Peripherals block to use UART 1 and I2C 1 peripherals, disabling other unwanted peripherals. Uncheck Timer 0. Enable FCLK_CLK1, the PL fabric clock and set its frequency either to 10.000 MHz for the Pynq

1. Select the MIO Configuration tab on the left to open the configuration form and expand I/O Peripheral in the right pane.
2. Click on the check box of the I2C 1 peripheral. Uncheck USB0, SD 0, ENET 0, GPIO > GPIO MIO as we don't need them.
3. Expand the Application Processing Unit group in the Select the MIO Configuration tab and uncheck the **Timer 0**.
4. Select the **Clock Configuration** in the left pane, expand the PL Fabric Clocks entry in the right, and click the check-box of *FCLK_CLK1*.
5. Change the Requested Frequency value of FCLK_CLK1 to **10.000 MHz**

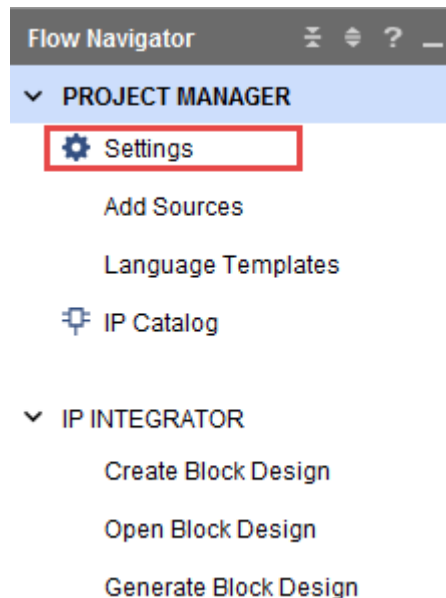


Enabling and setting the frequency of FCLK_CLK1

6. Click **OK**. Notice that the Zynq block only shows the necessary ports.

Add the provided I2C-based IP

1. In the Flow Navigator pane, click **Settings** under Project Manager.



Invoking Project Settings

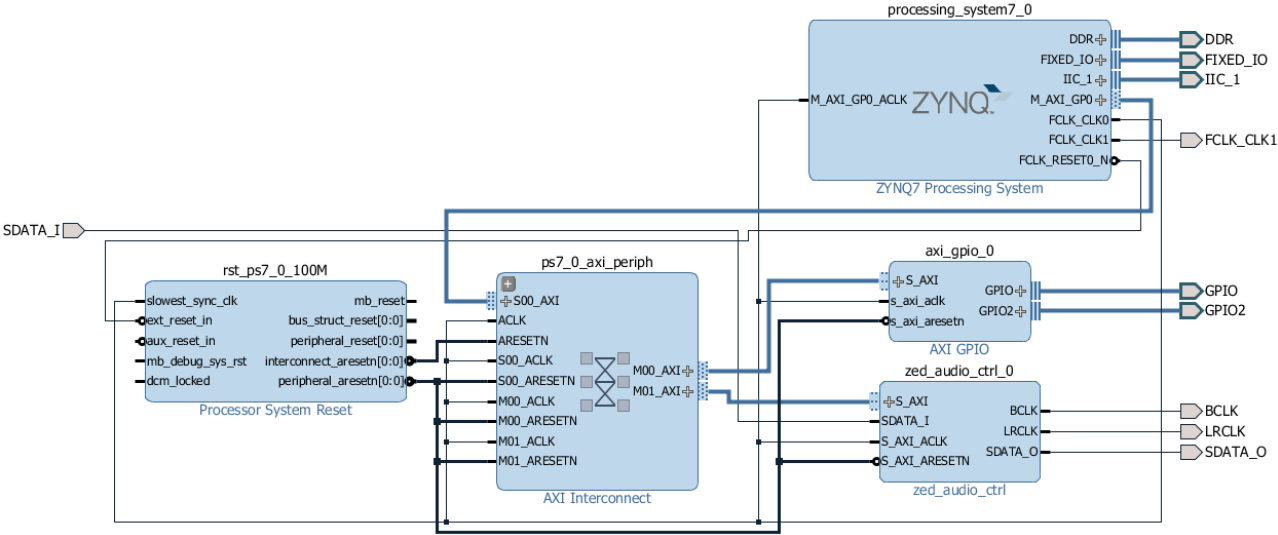
2. Expand **IP > Repository** in the left pane.
3. Click the + button. Browse to **c:\xup\hls\labs\lab4\ip_repo** and click **Select**. The directory will be scanned and added in the IP Repositories window, and two IP entry will be detected.
4. Click **OK** to accept the settings.

Instantiate zed_audio_ctrl and GPIO with width of 2 bits on channel 1 and width of 1 bit input only on channel 2. Run connection automation to connect them.

1. Click the **Add IP** button if the IP Catalog is not open and search for AXI GPIO in the catalog by typing *gpi* and double-click on the AXI GPIO entry to add an instance.
2. Click on the **Add IP** to Block Design button.
3. Double-click on the added instance and the Re-Customize IP GUI will be displayed.
4. Change the Channel 1 width to **2**.
5. Check the **Enable Dual Channel** box, set the width to *1* input only, and click **OK**.
6. Similarly add an instance of the **zed_audio_ctrl**.
7. Notice that Design assistance is available. Click on *Run Connection Automation*, and select **/axi_gpio_0/S_AXI**
8. Click **OK** to connect it to the M_AXI_GP0 interface. Notice two additional blocks, Proc Sys Reset, and AXI Interconnect have automatically been added to the design.
9. Similarly, click on *Run Connection Automation*, and select **/zed_audio_ctrl_0/S_AXI**.

Make IIC_1, GPIO, FCLK_CLK1, and zed_audio_ctrl ports external.

1. Select the GPIO interface of the *axi_gpio_0* instance, right-click on it and select **Make External** to create an external port. This will create the external port named GPIO and connect it to the peripheral.
2. Select the GPIO2 interface of the *axi_gpio_0* instance, right-click on it and select **Make External** to create the external port.
3. Similarly, selecting one port at a time of the *zed_audio_ctrl_0* instance, make them external.
4. Similarly, make the IIC_1 interface and FCLK_CLK1 port of the processing_system7_0 instance external. At this stage the design should look like shown below (you may have to click the regenerate layout button).



Block design after I2C based zed_audio_ctrl core added and connections made for the Pynq